Marine Plastic Tracker

MODEL · DATA · RESULTS

The model, the data, and what the numbers really say.

A plain-English walkthrough of the machine-learning side of the project: which model we trained, the five datasets we used and where they came from, how raw satellite files became training numbers, and exactly how accurate the result is — including the parts that did not work.

5	2.12 M	0.818	90 %
DATASETS	LABELLED PIXELS	MACRO-F1	HIGH-TIER PRECISION

Project: Marine Plastic Tracker — satellite plastic detection + drift forecasting for the Indian Ocean
Shipped model: `python_engine/models/marinenet.onnx` — 3-seed per-pixel MLP ensemble
Every number in this document was read from the project's own metrics files or re-measured live against the shipped model while writing it. Nothing is estimated.
Prepared for: project panel / viva · **Date:** 30 July 2026

SECTION 1

The 60-second answer

If the panel asks one question and you have one minute, this is the whole story.

What the model is

A **per-pixel neural network classifier** — technically an ensemble of three small multi-layer perceptrons (MLPs). It looks at **one 10 m × 10 m patch of sea at a time**, reads **11 numbers** describing the colour of that patch, and answers with one of **four labels**: **Plastic**, **Algae**, **Sea Foam**, or **Water**, plus a confidence score.

What it learned from

Real hand-labelled Sentinel-2 satellite data from published scientific benchmarks — **MARIDA** + **MADOS** (**1,081,615** training pixels, **532,566** validation, **499,907** completely unseen test pixels) plus **3,789** points from four **RFO** scenes. Two independent exams are held back entirely: RFO's Durban and Marmara scenes, and every **PLP** deployed-plastic target.

How well it works

On the 499,907 unseen pixels: **macro-F1 = 0.818**. For plastic specifically, **F1 = 0.799** (precision 0.776, recall 0.824). With our high-confidence filter switched on, plastic calls are **90 % precise** — 9 out of 10 flagged pixels really are debris.

Say macro-F1, never accuracy. Our raw accuracy is **0.9899**, which sounds spectacular and is actually meaningless: **98 % of ocean pixels are plain water**, so a model that answered "water" to everything would already score ~0.98. Macro-F1 averages the score of all four classes equally, so rare-but-important plastic counts as much as water. **0.818 is the honest number.**

What we built ourselves

The datasets are public, and PyTorch/NumPy/ONNX are libraries. Everything between them is ours: the four-class label mapping, the 11-feature spectral contract, the dataset readers, the loss function, the class-imbalance weighting, the decision-boundary calibration, the ensemble, the ONNX export with a bit-parity check, the two-tier confidence system, and the whole live inference path. **Section 10** is the itemised list, file by file.

The whole journey on one line

```
raw satellite files (GeoTIFF)  →  dataset adapter  →  11 features per pixel  →  .npz cache
        →  GPU training (PyTorch)  →  ONNX export  →  live engine (onnxruntime)  →  map markers
```

SECTION 2

What the model actually does

Before the datasets and the training, be crystal clear about the job. It is a very narrow job, and that is deliberate.

One pixel in, one label out

A Sentinel-2 satellite image is a grid. Each cell of that grid — each **pixel** — covers a square of sea **10 metres across**. The satellite does not record "colour" the way a phone camera does; it records **reflectance** in separate wavelength bands, including bands the human eye cannot see (near-infrared, short-wave infrared). Floating plastic reflects infrared light differently from water, and that difference is the entire physical basis of the project.

Our model takes the reflectance numbers for **one pixel** and outputs four scores that add up to 1 — how likely that pixel is Plastic, Algae, Sea Foam, or Water. The highest score wins.

```
INPUT  (11 numbers for one pixel)          OUTPUT (4 probabilities)
  B04  red reflectance                      Plastic    0.83   ←  winner
  B06  red-edge                             Algae      0.09
  B08  near-infrared                        Sea_Foam   0.05
  B8A  narrow near-infrared                 Water      0.03
  B11  short-wave infrared
  FDI  Floating Debris Index
  NDVI vegetation index
  PI   Plastic Index
  MNDWI water index
  NDVI_RE red-edge vegetation
  BRIGHT  overall brightness
```

Why four classes and not two

A naive "plastic vs. not-plastic" model floods the map with false alarms, because the two things that most look like plastic from space are **floating algae/sargassum** and **sea foam**. By forcing the model to name *which* of these it is seeing, we make it learn the differences instead of lumping every bright patch together. The four classes are our own mapping, condensed from the 15 fine-grained classes the source datasets ship with.

Why the last 4 features are free

Only the five band values (`B04 B06 B08 B8A B11`) are actually downloaded. The other six features are **arithmetic on those five** — published spectral indices that make the physics easier for a small network to learn. This matters practically: adding them cost us **zero extra satellite data and zero extra download time**.

FEATURE	FORMULA	WHAT IT HIGHLIGHTS
FDI Floating Debris Index	`B08 − [B06 + (B11 − B06) × 0.1172 × 10]`	Anything floating on the surface — the primary debris signal (Biermann et al. 2020)
NDVI	`(B08 − B04) / (B08 + B04)`	Living vegetation → separates algae from plastic
PI Plastic Index	`B08 / (B08 + B04)`	Plastic-specific NIR/red balance
MNDWI	`(B08 − B11) / (B08 + B11)`	Water vs. floating matter
NDVI_RE	`(B8A − B06) / (B8A + B06)`	Red-edge vegetation → sargassum
BRIGHT	`mean of all 5 bands`	Foam and sun-glint, which are simply bright

The contract rule. These 11 numbers must be computed in *exactly* the same order and with exactly the same formulas during training and during live scanning — otherwise the model silently receives garbage. We enforce this by having one function in the trainer (`augment11` in `train_perpixel_gpu.py`) mirrored line-for-line by one function in the engine (`_features` in `inference_engine.py`), with the order also frozen as a constant in `model_loader.py` . This is the single most common way an ML project quietly breaks, and we designed against it.

SECTION 3

The five datasets — and where they came from

We work with **five datasets**. Three contribute training data, two provide exams the model never learns from, and one is a synthetic sanity-check baseline we generate ourselves. (RFO does both: four of its six scenes train, two are held out.) Every real one is a published, citable scientific dataset — none of it is scraped or invented.

#	DATASET	SOURCE & CITATION	WHAT IS IN IT	OUR ROLE FOR IT
1	**MARIDA** Marine Debris Archive	Kikaki, Karantzalos et al., *PLOS ONE* 17(1):e0262247, **2022**. Public research release.	Hand-labelled Sentinel-2 patches: 11-band reflectance GeoTIFFs + a pixel-level class mask, 15 fine classes, with **official train/val/test split files**. **1,381 patches** (694 train / 328 val / 359 test).	**TRAIN + TEST**
2	**MADOS** Marine Debris & Oil Spill	Kikaki, Kakogeorgiou, Hoteit, Karantzalos, *ISPRS J. Photogramm. Remote Sens.*, **2024**. Zenodo DOI **10.5281/zenodo.10664073**. (~4.0 GB download.)	The same group's successor to MARIDA. **174 scenes**, sparse pixel-level annotation, 15 thematic classes, ACOLITE surface reflectance. **2,803 crops** (1,433 / 642 / 728).	**TRAIN + TEST**
3	**RFO** RefinedFloatingObjects	Rußwurm et al., **2023**. Public research release.	**6 full Sentinel-2 scenes** from six different seas — Accra, Durban, Lagos, Marmara, New Orleans, Venice — with **4,689** hand-refined point annotations (debris = 1, water = 0).	**TRAIN (4 scenes) + EXAM (2 scenes)** Durban & Marmara held out
4	**PLP** Plastic Litter Project 2021/2022	Topouzelis et al., **University of the Aegean**. Public research release.	The only **true ground truth** available anywhere: researchers physically floated **real HDPE and PVC plastic targets** at known coordinates on known dates and had Sentinel-2 photograph them — plus **wooden decoys** as controls. **21 target dates.**	**HELD-OUT EXAM** hardest possible test
5	**Synthetic spectral library**	**Ours** — generated by `ml/datasets/dataset.py`	Physics-derived fake pixels for each class, from published reflectance ranges.	**BASELINE ONLY** never shipped

> **Why four real datasets instead of one?** Each one closes a hole the others leave open. MARIDA gives clean official splits. MADOS triples the volume and adds scene variety. RFO adds **globally diverse water and debris examples from six different seas** — and its two withheld scenes answer "does it work in a sea it has never seen?" PLP answers "does it find *actual verified plastic*?" A project with only MARIDA can be accused of overfitting one benchmark; with all four, the weaknesses are measured rather than guessed at.

How much labelled data actually reached the model

Not every pixel in these datasets is usable. Only pixels the researchers explicitly labelled count, and we deliberately discard some classes (ships, clouds, waves, wakes, cloud shadows, oil spills, jellyfish, sea snot) because they are not part of our four-class job and would only add noise. After that filtering:

SPLIT	SOURCE	TOTAL PIXELS	PLASTIC	ALGAE	SEA FOAM	WATER
Train	MARIDA	347,204	1,943	2,684	469	342,108
	MADOS	734,411	2,561	9,548	469	721,833
	RFO (4 scenes, points)	3,789	1,549	—	—	2,240
	combined	**1,085,404**	6,053	12,232	**938**	**1,066,181**
Validation	MARIDA	187,163	1,075	1,644	369	184,075
	MADOS	345,403	1,162	4,462	369	339,410
	combined	**532,566**	2,237	6,106	**738**	**523,485**
Test never seen	MARIDA	153,762	381	1,690	387	151,304
	MADOS	346,145	973	5,638	387	339,147
	combined	**499,907**	1,354	7,328	**774**	**490,451**
Grand total		**2,117,877**	plus 900 held-out RFO points (Durban, Marmara) and 21 PLP target dates held aside as exams			

Look at the imbalance — it drives every design decision that follows. In training there are **1,063,941 water pixels and 938 sea-foam pixels**: a ratio of about **1,134 : 1**. Left alone, gradient descent simply learns "always say water". Section 7 explains the three specific techniques we used to defeat this.

Supporting data used elsewhere in the app but **not** for training the classifier: Copernicus Marine (CMEMS) ocean-current and wind fields for the 72-hour drift forecast, and a Natural Earth coastline GeoJSON for shoreline-impact estimates.

SECTION 4

How we got the datasets into the project

This is the question panels ask when they suspect a downloaded notebook. The honest answer is that the download is the easy 5 %; the work is the **adapter layer** we wrote to turn four completely different file formats into one identical numeric contract.

STEP 1
Download
Fetch each dataset from its published source (Zenodo / research page) as an archive.

STEP 2
Unpack
Extract into a fixed folder under `data/`, one folder per dataset, untouched.

STEP 3
Adapter
Our Python reader opens the GeoTIFFs/shapefiles and emits the same 7 numbers per pixel.

STEP 4
Cache
Save as compressed `.npz` arrays so retraining never re-reads a GeoTIFF.

STEP 5
Train
The GPU trainer loads only `.npz` — no GDAL, no rasterio, instant startup.

Where things live on disk

```
marine-plastic-tracker/
├── data/                                  ← raw datasets, exactly as published
│   ├── marida/        patches/  splits/  shapefiles/  labels_mapping.txt
│   ├── MADOS/         Scene_0 … Scene_173/  splits/
│   ├── refinedfloatingobjects/             6 scenes + point shapefiles
│   └── plp/PLP/       PLP2021/  PLP2022/   Sentinel-2 scenes + target shapefiles
│
└── python_engine/
    ├── ml/datasets/                       ← THE ADAPTERS (written by us)
    │   ├── marida.py    reads 11-band GeoTIFF + class mask
    │   ├── mados.py     reads per-wavelength bands, resamples 20 m/60 m → 10 m
    │   ├── rfo.py       reads full scenes + point shapefiles, scene-disjoint split
    │   ├── plp.py       reprojects deployed-target coordinates into scene pixels
    │   └── dataset.py   synthetic baseline library
    ├── ml/train_perpixel_gpu.py            ← THE TRAINER
    └── models/                            ← caches + trained weights
        ├── marida_{train,val,test}.npz      cached feature/label arrays
        ├── mados_{train,val,test}.npz
        ├── rfo_pixels_{train,test}.npz      held-out exam
        ├── plp_pixels_{train,test}.npz      held-out exam
        ├── marinenet.onnx                   ← THE SHIPPED MODEL (551 KB)
        └── metrics.json                     ← the published model card
```

34 GB in, 35 MB out. The raw datasets on disk total about **34 GB** of satellite imagery (RFO 23 GB, MADOS 5.6 GB, MARIDA 4.5 GB, Durban 0.9 GB, PLP 79 MB). What the classifier actually trains on is **34.9 MB** of `.npz` feature arrays — a roughly **1,000× reduction**. That gap *is* the preprocessing: out of tens of gigabytes of pixels, only the ones a researcher explicitly labelled, in the five bands we can fetch live, reduced to 7 numbers each, survive. It is also why retraining takes minutes instead of hours — no GeoTIFF is ever re-read.

Why each adapter was real work, not boilerplate

DATASET	THE PROBLEM IN THE RAW FILES	WHAT OUR ADAPTER DOES ABOUT IT
MARIDA	Bands are stacked in one 11-band file; labels are 1–15 in a separate mask file; patch names encode their folder (`11-6-18_16PCC_13` → `S2_11-6-18_16PCC/`).	Resolves the filename convention, indexes out the 5 bands we need by position, and applies our 15→4 class map.
MADOS	Every band is a **separate file named by wavelength** (`..._rhorc_833_0.tif`), stored in `10/` , `20/` , `60/` folders at three different **pixel sizes**. And the class IDs are **renumbered** vs. MARIDA.	Sorts files by wavelength to recover MARIDA's band order, nearest-neighbour upsamples the 20 m and 60 m bands onto the 10 m grid, and — importantly — we **verified the class renumbering empirically** instead of copying MARIDA's map (see the note below).
RFO	Six scenes from different years vary between 12-band and 13-band, L1C and L2A, and values are stored as integers (0–10000), not reflectance.	Resolves band position per file via its embedded descriptions, divides by 10,000, and does one windowed read per labelled point. Splits by **scene**, not randomly.
PLP	Targets are given as map coordinates in a shapefile with its own projection — not pixels. Some scenes are integer-scaled, some already reflectance.	Reads the shapefile's `.prj` , builds a coordinate transformer into the scene's projection, converts to a row/column, and reads that one pixel. Decides the scaling factor **once per scene** from a whole-scene median.

A real bug we found and fixed — worth telling the panel. Our first PLP reader decided integer-vs-reflectance scaling *per pixel*. Dark water pixels read below the threshold even in an integer-scaled scene, so the divide-by-10,000 was skipped and raw integers landed in the cache — `B11 ≈ 158` , `FDI ≈ −185` , physically impossible values poisoning our hardest test set. Fix: decide the factor **once per scene** from a coarse whole-scene median, which cleanly separates the two cases. This is the kind of silent data bug that inflates or destroys results without any error message.

We did not trust the documentation. MADOS class 6 is documented as "Ships". It holds 140,000 pixels — far too many for ships. So we wrote a diagnostic (`class_signatures()`) that prints the **mean spectral signature of every raw class**. Class 6 came back with `FDI 0.0117` , `NDVI −0.09` and water-like brightness: unambiguously water background, not ships. We mapped it to Water. Had we copied MARIDA's map blindly, 140,000 water pixels would have been mislabelled.

SECTION 5

Preprocessing, step by step

This is the pipeline that turns a folder of satellite GeoTIFFs into two NumPy arrays — `x` of shape **[n, 11]** and `y` of shape **[n]**. Nothing more exotic than that is needed, but every step has a reason.

Step 1 — Open the scene and its label mask

Read the reflectance array and the researcher-drawn class mask, and crop both to their common overlap (they occasionally differ by a row or column).

Step 2 — Put every dataset on one 10 m grid

Sentinel-2 records different bands at different resolutions (10 m, 20 m, 60 m). Our features mix bands across resolutions, so all bands are **nearest-neighbour upsampled to 10 m**. Nearest neighbour, not bilinear — interpolation would **invent** reflectance values that were never measured, and a fake bright pixel next to a real debris pixel is exactly the kind of false positive we are trying to eliminate.

Step 3 — Select 5 bands out of 11

We keep only `B04, B06, B08, B8A, B11`. Not to save space — because **the live satellite pipeline only downloads these five**. Training on bands we cannot fetch at run time would produce a model that is useless in production. The training contract is deliberately handcuffed to the operational contract.

Step 4 — Fix the scale to reflectance 0–1

Some providers ship integers 0–10,000; some ship reflectance 0–1. Everything is normalised to 0–1 so a pixel means the same thing regardless of which dataset it came from.

Step 5 — Compute the features

Apply the FDI and NDVI formulas → **7 base features**. Later, in the trainer, apply `augment11()` → **11 features**. Both stages are pure arithmetic on the five bands.

Step 6 — Map 15 research classes down to our 4

OUR CLASS	MARIDA SOURCE CLASSES	MADOS SOURCE CLASSES
Plastic (0)	Marine Debris	Marine Debris
Algae (1)	Dense Sargassum, Sparse Sargassum, Natural Organic Material	Dense Sargassum, Sparse Floating Algae, Natural Organic Material
Sea Foam (2)	Foam	Foam
Water (3)	Marine Water, Sediment-Laden Water, Turbid Water, Shallow Water, Mixed Water	Marine Water, Sediment-Laden Water, Turbid Water, Shallow Water, + verified class 6
Dropped	Ship, Clouds, Waves, Cloud Shadows, Wakes	Oil Spills, Oil Platforms, Sea Snot, Jellyfish, Waves & Wakes

> **One mapping choice is our best anti-false-positive trick.** Sending **Sediment-Laden and Turbid Water → Water** is deliberate. Muddy monsoon coastal water is *bright* and produces a high FDI, so a naive index-threshold detector screams "plastic" all along the Indian coast in monsoon season. By feeding thousands of labelled turbid-water pixels in as **Water**, we teach the network directly that bright brown water is not debris.

Step 7 — Drop non-finite pixels

Any pixel whose 7 features are not all finite (division by zero, nodata gaps, sensor artefacts) is dropped: `np.isfinite(px).all(axis=1)`. One `NaN` reaching a gradient turns every weight in the network to `NaN`.

Step 8 — Respect the official splits

Both MARIDA and MADOS ship **official train/val/test split files**, and we use them verbatim rather than shuffling all pixels randomly. This matters enormously: neighbouring pixels in one scene are almost identical, so a random split leaks the test set into training and inflates every score. RFO is split by **scene** (train: Accra, Lagos, New Orleans, Venice — test: Durban, Marmara) and PLP by **date** (earlier dates train, latest 30 % test), for the same reason.

Step 9 — Cache to .npz

The result is saved compressed. This is not just a speed trick — it **decouples the environments**. Reading GeoTIFFs needs rasterio/GDAL; training needs a CUDA PyTorch build. Getting both into one Python environment on Windows is painful. Caching means the GPU trainer only ever loads `.npz` files and needs neither.

Step 10 — Standardise inside the model, not outside

Neural networks want inputs centred near zero. Rather than normalising in a preprocessing script the engine would then have to replicate exactly, we store the training mean and standard deviation as **buffers inside the network** and subtract them in `forward()`. When exported, the ONNX file therefore **consumes raw features**. There is no external normalisation step that could drift out of sync — a whole class of production bug is designed out of existence.

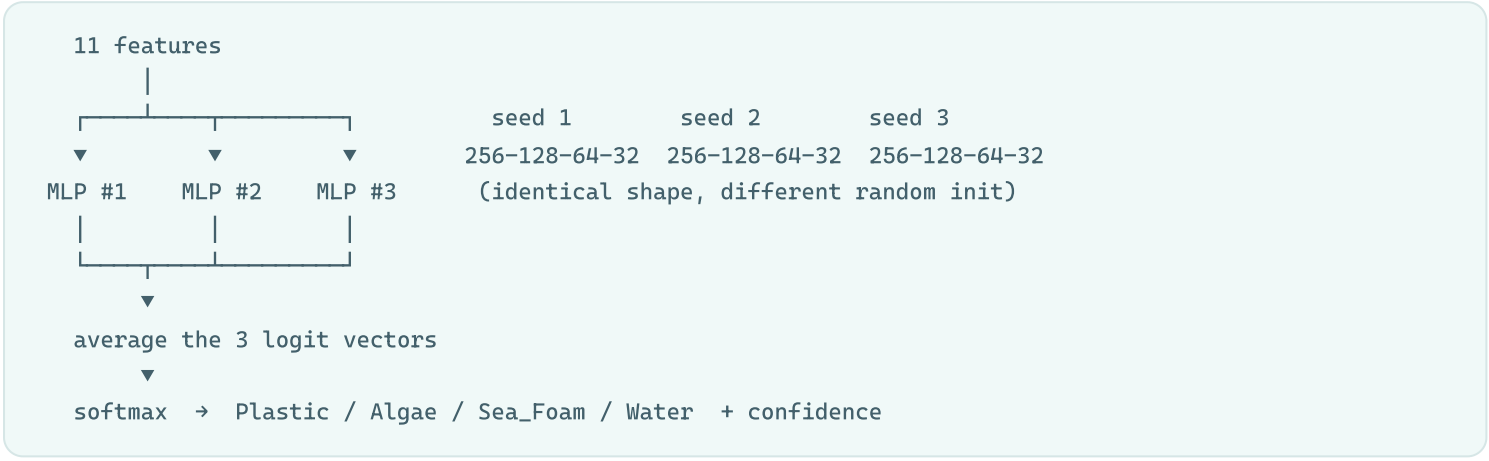
```
GeoTIFF → 10 m grid → 5 bands → reflectance 0–1 → 7 features → 15→4 labels
        → drop NaN → official split → .npz cache → augment11 → [n, 11] + [n]
```

SECTION 6

The model we trained

Architecture

Three independent **multi-layer perceptrons**, each with layer widths **11 → 256 → 128 → 64 → 32 → 4**, ReLU activations and dropout between hidden layers. At inference all three run and their raw outputs (logits) are **averaged** before the winner is picked. About **141,000 parameters** total; the exported file is **551 KB**.



Why an MLP and not a CNN — we tested it, we did not assume it

The obvious choice for satellite imagery is a convolutional network that sees spatial context. **We built one** — a residual ResNet-style CNN on 64×64×6 patches, GPU-trained (`train_patch_cnn.py`) — and evaluated it on the *identical* held-out test. It lost, badly and instructively:

MODEL	TRAIN ACCURACY	HELD-OUT MACRO-F1	DIAGNOSIS
Patch CNN (ResNet, 64×64×6)	0.9872	0.555	**Memorised scenes.** Huge train–test gap: with only a few hundred distinct acquisitions, 64×64 texture identifies *the scene*, not the plastic.
Per-pixel MLP ensemble	0.9899	**0.818**	The spectral signature of plastic is a **physical** property that transfers to any sea; texture is not.

The CNN is kept in the repository behind an opt-in environment variable (`CNN_MODEL_PATH`) and its 0.555 is published in `metrics.json` . It is a **reported negative result**, not a hidden failure — and it is the single strongest evidence that our architecture choice was measured rather than guessed.

The second stage: Clay v1.5 as a verifier

We also wired in **Clay v1.5**, an open geospatial foundation model. Its encoder is **frozen** — we do not retrain it — and we trained only a small `1024 → 256 → 4` head on top of the 1024-dimensional embeddings it produces for a 64×64 chip.

ON IDENTICAL HELD-OUT CHIPS	CLAY + HEAD	OUR MLP	SCRATCH CNN
macro-F1	**0.711**	0.690	0.555
Debris recall, brand-new seas (Durban + Marmara)	**0.542**	0.222	—
Water false-positive rate on those seas	7.6 %	**0.4 %**	—

Read that table carefully, because it explains our design. A pretrained backbone **fixes the scene-overfitting that killed our own CNN** and catches **2.4× more debris in seas it has never seen** — but it is far more trigger-happy over open water, and its own plastic F1 is only 0.37. So Clay is **not** the detector. Our MLP detects; Clay **confirms**. When both agree that a patch is plastic, measured precision rises to **0.92**. Two independent methods agreeing is a stronger claim than either alone.

SECTION 7

How training actually worked

Hardware: one **NVIDIA RTX 4050 laptop GPU**. Framework: PyTorch, in a separate conda environment (`mpt-train`) that is kept entirely out of the production engine.

The basic loop

1. Load the cached `.npz` arrays — MARIDA + MADOS training splits plus the 3,789 RFO points from the four training scenes — and compute the 11 features.
2. Compute the training mean/std and store them **inside** the model.
3. For each epoch: shuffle, walk through in mini-batches of **8,192** pixels, forward pass, compute loss, backpropagate, step **AdamW** (learning rate 2×10^{-3}, cosine decay, weight decay 10^{-4}), with mixed-precision on the GPU.
4. After every epoch, score **macro-F1 on the validation split** and keep a snapshot if it improved.
5. At the end, **restore the best-validation epoch** — not the last one.

> **Point 4 and 5 are the honesty mechanism.** Epoch selection uses **macro-F1**, not accuracy, on the **validation** split — never the test split. The 499,907 test pixels are touched exactly once, at the very end. Nothing about them influenced any choice.

The three techniques that beat the 1,134 : 1 imbalance

1. Focal loss written by us, 8 lines

Ordinary cross-entropy gives every pixel an equal vote, so a million easy water pixels drown out four thousand hard plastic ones. Focal loss (Lin et al. 2017) multiplies each pixel's loss by `(1 − p_correct)^γ` with **γ = 1.5**. A water pixel the model already gets right with 99 % confidence contributes almost nothing; a plastic pixel it is unsure about contributes almost fully. **Attention is redirected to the hard, rare classes.**

2. Class-balanced weights written by us

Each class is additionally weighted by the **effective number of samples** (Cui et al. 2019): `w_c ∝ (1 − β) / (1 − β^n_c)` with **β = 0.999**. This is deliberately gentler than plain inverse-frequency, and we chose it after being burned: full inverse-frequency weighting made an earlier model flag Sea Foam at **1.6 % precision** — a flood of false positives.

3. Folded bias calibration written by us

Even a well-trained network's raw *argmax* is not the macro-F1-optimal decision boundary under extreme imbalance. So after training we run **coordinate ascent** over a small grid of per-class logit offsets, keeping whatever maximises **validation** macro-F1 — then **add those offsets into the final layer's bias vector**. The exported model needs no threshold logic at run time: plain argmax is already calibrated. This was the single largest macro-F1 lever we found.

4. Three seeds, averaged

Train the same architecture three times from different random initialisations and average the logits. Each network makes slightly different mistakes; averaging cancels the uncorrelated ones. **Nothing is free** — see the trade below.

The trade we made on purpose

A single calibrated 11-feature model actually scored a **higher headline macro-F1** than the ensemble. We shipped the ensemble anyway:

METRIC	SINGLE MODEL	3-SEED ENSEMBLE (SHIPPED)	VERDICT
macro-F1 (headline)	0.8242	0.8175	slightly worse ✗
Plastic F1	0.742	**0.799**	better ✓
Plastic precision	0.652	**0.776**	much better ✓
False plastic over open water, new seas	3.0 %	**0.7 %**	4× better ✓

The reasoning is operational, not statistical. This system dispatches boats and crews. A false alarm sends a real crew to empty water and burns credibility; a slightly lower average score across four classes costs nothing operationally. We optimised for **plastic precision** and accepted 0.007 of macro-F1 to get **four times fewer false alarms**. That choice, and the fact that we can show the numbers on both sides of it, is the point.

Export, and proving the export is faithful

The trained PyTorch model is exported to **ONNX** (opset 13) so the production engine can run it with `onnxruntime` and **never needs PyTorch installed** — the engine is numpy + onnxruntime only. Export can silently change numerics, so we verify: push 256 random vectors through both PyTorch and the exported ONNX and compare predictions. **Agreement = 1.0000**, recorded in `metrics.json`. The thing we measured is *exactly* the thing that ships.

```
$ conda run -n mpt-train python -m ml.train_perpixel_gpu \
      --epochs 80 --loss focal --weights cb --hidden 256,128,64,32 --calibrate
```

Nothing is promoted automatically. The trainer writes candidates (`marinenet_pp7.onnx`, `marinenet_pp11.onnx`) plus a head-to-head verdict against the currently shipped model on an identical test set. A human reads the verdict and copies the winner over `marinenet.onnx`. Every superseded model is kept (`marinenet_pre_*.onnx`) so any claim can be re-checked.

SECTION 8

Results — how accurate is it, really

All figures below are on the **499,907 held-out pixels** the model never saw during training or model selection. They come from `python_engine/models/metrics.json` and were re-measured against the shipped ONNX file while this document was written.

Headline

METRIC	VALUE	HOW TO READ IT
Macro-F1	**0.8175**	**Lead with this.** Average F1 across all four classes, each counting equally.
Accuracy	0.9899	**Water-inflated — always caption it.** Water is 98 % of pixels; "always water" scores ~0.98.
Balanced macro-F1	0.8776	Same metric on a class-balanced subsample (Water capped at 4,000 pixels).
Test pixels	499,907	Official MARIDA + MADOS test splits.
ONNX ↔ PyTorch agreement	1.0000	The exported file is the model we measured.

Per class

CLASS	PRECISION	RECALL	F1	IN PLAIN WORDS
Plastic	0.776	0.824	**0.799**	**Of every 100 pixels called plastic, ~78 really are. Of all real plastic present, it finds ~82 %.**
Algae	0.648	0.822	0.725	Finds most algae but over-calls it — the weakest class.
Sea Foam	0.681	0.835	0.750	Respectable given only **938** training examples exist.
Water	0.998	0.993	0.995	Near-perfect — and the reason accuracy alone is misleading.

Confusion matrix — where the mistakes actually go

Rows = ground truth · Columns = model's prediction · green diagonal = correct

	→ PLASTIC	→ ALGAE	→ FOAM	→ WATER	TOTAL
Plastic	**1,116**	93	10	**135**	1,354
Algae	**186**	**6,024**	142	976	7,328
Sea Foam	**48**	10	**646**	70	774
Water	**88**	3,163	150	**487,050**	490,451

Three things a panel should notice:

- **Only 88 of 490,451 water pixels** were called plastic — a **0.018 %** false-alarm rate on the class that dominates every real scan. This is the number that makes the system usable.
- The main plastic confusion is **135 plastic pixels called water** (missed, usually faint or partly-filled pixels) and **186 algae pixels called plastic** — the physically hardest confusion, since sargassum and plastic genuinely overlap spectrally.

- The largest single error, **3,163 water pixels called algae**, is a nuisance rather than a risk: it depresses algae precision but never dispatches a crew, because only *plastic* drives hotspots and dispatch.

Per dataset — and why they differ

TEST SET	PIXELS	ACCURACY	MACRO-F1	PLASTIC F1
MARIDA	153,762	0.9981	0.9047	0.822
MADOS	346,145	0.9862	0.7843	0.791
Combined (reported)	**499,907**	0.9899	**0.8175**	0.799

MADOS is harder — sparser annotation, more scenes, messier coastal water. **We report the combined figure, which is the lower one.** Quoting MARIDA's 0.905 alone would be cherry-picking.

The two-tier confidence system

A single model gives one number; operations needs two answers — "show me everything that might be plastic" and "show me only what I would send a boat to". So we swept the plastic-probability threshold across the held-out test and published the trade-off curve. These rows were **re-measured live** for this document:

THRESHOLD ON P(PLASTIC)	PIXELS FLAGGED	PRECISION	RECALL	USED FOR
argmax (no filter)	1,438	0.776	0.824	Nothing — too noisy for dispatch
≥ 0.50	1,220	0.838	0.755	—
≥ 0.648 → "HIGH"	850	**0.901**	0.566	**Hotspots and crew dispatch use this tier only**
≥ 0.79	232	0.948	0.163	Available if a mission needs near-certainty

0.648 is not a guess — it is the threshold at which measured precision crosses 90 %. Everything below it still appears on the map, clearly labelled as a lower-confidence detection. **We trade recall for precision on purpose**, because the cost of a wasted patrol is much higher than the cost of missing one patch that the next 5-day satellite pass will likely catch again.

Adding Clay: the confirmation tier

DECISION	PRECISION	RECALL	SHOWN IN THE APP AS
MLP plastic, HIGH tier	0.901	0.566	Plastic detection
MLP + Clay both agree	**0.922**	0.704	**CONFIRMED**
MLP plastic, Clay disagrees	—	—	POSSIBLE (downgraded, still visible)

SECTION 9

The independent exams — including the one we fail

MARIDA and MADOS are the benchmarks we trained on, so good scores there are necessary but not sufficient. **Durban and Marmara** (RFO) and **every PLP target** were never trained on. Both results are published in-app, not buried.

Exam 1 — RFO: two seas the model has never seen

RFO's six scenes are split **by scene**, not randomly: Accra, Lagos, New Orleans and Venice contribute training points; **Durban and Marmara** are withheld entirely — 900 hand-refined points in two unfamiliar bodies of water. Measured live for this document:

METRIC	RESULT	READING
False plastic over open water	**0.74 %**	**The critical number. In two unfamiliar seas it stays calm: ~7 false alarms per 1,000 water points.**
Debris recall	30.7 %	It finds under a third of the flagged debris. Honest and expected — see below.

Why recall drops from 82 % to 31 %: RFO's "debris" points are **mixed floating matter** — wood, sargassum and litter together — while we trained on "Marine Debris". Missing a floating log is arguably correct behaviour for a plastic detector. And crucially, the failure mode is **conservative**: it misses things rather than inventing them.

Why RFO's training scenes are in the model — measured, by ablation

We did not simply assume the extra data helped. We trained the **identical ensemble recipe twice** — same seeds, same configuration, RFO in versus RFO out — and scored both on the same held-out test:

ENSEMBLE VARIANT	MACRO-F1	PLASTIC PRECISION	PLASTIC RECALL	PLASTIC F1
Without RFO training points	0.747	0.694	0.901	0.784
With RFO (shipped)	**0.818**	**0.776**	0.824	**0.799**

The mechanism is worth stating precisely: RFO's value is not more plastic examples — it is **diverse global water *negatives*** from six unrelated seas. Those sharpen the water/plastic boundary, which is why **precision** rises by 8 points while recall drops slightly. More negatives, not more positives, is what bought the reliability.

Historical note for completeness: an *earlier, pre-ensemble* single model that added RFO lost ground on the primary benchmark (0.787 → 0.767) and was rejected at the time — that experiment is still recorded in `metrics.json` under `rfoExperiment`. It was the later ensemble recipe, with focal loss and calibration, that turned RFO into a clear win. Both records are kept, so the decision trail is auditable rather than retrofitted.

Exam 2 — PLP: real plastic, physically deployed. We score 0 / 33.

> **Published, not hidden.** The Plastic Litter Project floated real HDPE and PVC targets in the Aegean at known coordinates on known dates. On the date-disjoint held-out split — **33 pixels of genuinely verified plastic** — our shipped model detects **0**.
>
> It also raised **0 false plastics on the 120 held-out water pixels** from the same scenes, which tells you the model is not broken; it simply cannot see these targets.

Why, physically: the targets are **3.5–14 m across**. A Sentinel-2 pixel is **10 m × 10 m = 100 m²**. A 28 m² target fills roughly a quarter of one pixel, so the pixel the satellite records is **~75–90 % water**. The plastic signature is diluted below anything a spectral classifier can recover. This is a **resolution limit of the sensor**, not a defect in our training.

And we proved it is a resolution limit rather than a data gap, by testing the obvious objection — "just train on PLP too":

MODEL	PLASTIC PRECISION	PLP TARGETS FOUND	DECISION
Shipped (no PLP in training)	**0.776**	0 / 33	Kept
Retrained with PLP included	0.660	2 / 33	**Rejected**

Teaching the network that water-like spectra are "plastic" bought **2 extra sub-pixel targets** and cost **12 points of plastic precision** — thousands of new false positives across a real scan. We did not promote it. **Reporting a tested negative result is stronger science than hiding it**, and we would rather the panel hear it from us with the numbers than discover it themselves.

A looser variant of the same test — allowing a 3-pixel search window around each target across all 21 dates — does register a plastic hit on **7 of 21 dates (33 %)**. Both figures are recorded in `metrics.json`. We lead with the strict 0/33 because it is the less flattering one.

The ceiling, stated plainly

> **Plastic F1 cannot be pushed to 0.95 with Sentinel-2 spectral data, by us or by anyone.** Plastic overlaps algae, foam and wet wood spectrally, and real debris is frequently sub-pixel at 10 m. The remaining levers are not better training recipes — they are **higher-resolution imagery** (PlanetScope 3 m, drones), **SAR**, or **multi-temporal fusion**: different projects, not tuning runs. So the honest claim is not "we detect plastic better than the field". It is: **when we say HIGH-confidence plastic, we are right about 9 times out of 10, and we can show you exactly how we know that.** We sell precision, and transparency about the limits.

Other limits we disclose in the app rather than in a footnote:

- The drift forecast's physics harness exists (Liu–Weisberg skill score) but has **not yet been validated against real drifter tracks**, so ETAs are labelled as estimates.
- The biofouling correction is a **labelled heuristic**, and is off by default.
- Satellites pass every ~5 days and cannot see through cloud. Each pixel is 10 m, so only **large patches** are visible — never a single bottle. The app states this on its own landing page.

SECTION 10

What we implemented ourselves

The fair question is: where does the library end and the project begin? Here is the line, itemised. OURS = code we wrote. LIBRARY = something standard we used.

COMPONENT	WHO	WHAT IT INVOLVED	FILE
Four-class label mapping	OURS	Condensing 15 research classes to 4 for both datasets, including the anti-false-positive turbid-water decision and **empirically validating** MADOS's renumbering by mean spectra rather than trusting the docs.	`datasets/marida.py`, `datasets/mados.py`
11-feature spectral contract	OURS	Choosing the features, implementing FDI/NDVI/PI/MNDWI/NDVI_RE/BRIGHT, and mirroring the exact order in three places so training and inference can never diverge.	`augment11()`, `inference_engine._features`, `model_loader.FEATURE_NAMES`
Four dataset adapters	OURS	Wavelength-sorted band stacking, multi-resolution resampling, coordinate reprojection of deployed targets, per-scene reflectance scaling, scene- and date-disjoint splitting, `.npz` caching.	`ml/datasets/*.py`
Focal loss	OURS	Implemented directly in ~8 lines with optional label smoothing — not imported.	`train_perpixel_gpu.py:106`
Class-balanced weights	OURS	Effective-number-of-samples weighting, with the `β` chosen after a documented failure of plain inverse-frequency weighting.	`cb_weights()`
Bias calibration	OURS	Coordinate-ascent search over per-class logit offsets against validation macro-F1, then folded into the final layer so the export stays a plain argmax drop-in.	`calibrate_bias()`
Metrics + evaluation harness	OURS	Confusion matrix, per-class precision/recall/F1, macro-F1, balanced-subsample view, per-dataset breakdown, automatic head-to-head verdict against the shipped model.	`_confusion`, `_prf`, `_macro_f1`, `report()`
Ensembling & promotion policy	OURS	Three seeds averaged into one exported graph; nothing auto-promoted; every superseded model retained for re-checking.	`models/marinenet*.onnx`
ONNX export + parity proof	OURS	Export with dynamic batch axis, plus a 256-vector PyTorch-vs-ONNX agreement check (= 1.0000) recorded in the model card.	`export_onnx()`
Hand-written MLP baseline	OURS	A complete **pure-NumPy** MLP — forward pass, manual backpropagation, Adam optimiser hand-implemented, standardisation algebraically folded into the first Gemm, and the ONNX graph **hand-constructed node by node** with `onnx.helper`. No autograd anywhere. Written to prove we understand what PyTorch does for us.	`ml/train.py`
Two-tier confidence system	OURS	Threshold sweep on held-out data, 0.648 chosen where precision crosses 90 %, wired through to hotspot clustering and dispatch so only HIGH-tier plastic can send a crew.	`config.py:63`, `batch_classifier.py`, `main_pipeline.py:89`
Clay verifier path	OURS	Frozen-encoder head training, embedding cache, a **pure-NumPy** cosine similarity index (so the production engine needs no PyTorch), plastic-affinity and novelty scores, and the confidence-fusion rule calibrated to 0.92 precision.	`clay_*.py`, `similarity_service.py`, `confidence_fusion.py`
Production inference path	OURS	Lazy thread-safe model loading (so two concurrent scans can't observe a half-loaded model and silently fall back), automatic CUDA provider with CPU fallback, a `REQUIRE_MODEL` strict mode that refuses to run un-	`model_loader.py`, `inference_engine.py`

		benchmarked heuristics, and a model card served over HTTP to the UI.	
Neural network primitives, autograd, CUDA kernels	LIBRARY	PyTorch (training only), NumPy, onnxruntime (serving), rasterio/GDAL (GeoTIFF reading), pyproj (reprojection).	—
Datasets and the Clay backbone	LIBRARY	Published research artefacts, cited. We wrote every line that reads, cleans, labels, splits, trains on, evaluates and serves them.	—

SECTION 11

How the model gets its data when the app is running

Training is offline and finished. This is what happens live when an analyst draws a box on the map and presses **Scan** — the same 11 features, computed by the same formulas, on fresh satellite pixels instead of cached ones.

STAGE 1
Find a scene
Query Copernicus for Sentinel-2 passes over the box; pick the least-cloudy composite under 50 % cloud.

STAGE 2
Download 5 bands
Fetch `B04 B06 B08 B8A B11` at native 10 m resolution.

STAGE 3
Mask
Drop cloud, cirrus, shadow, snow, land and nodata pixels via the scene's own SCL layer.

STAGE 4
FDI screen
Compute FDI for every surviving pixel; keep only `FDI > 0.02` as candidates.

STAGE 5
Classify
Build the 11 features per candidate → `marinenet.onnx` → class + confidence.

STAGE 6
Cluster
Group **HIGH-tier plastic only** into hotspots and score severity.

STAGE 7
Forecast
Push each hotspot 72 h forward on real currents + wind; raise alerts.

Why the FDI screen comes before the model. A modest scan area contains **millions** of pixels. Running a network on every one is wasteful, and the physics already tells us that debris must sit above the water baseline in the near-infrared. FDI is a cheap, published, interpretable filter that reduces millions of pixels to a handful of candidates; the network then does the genuinely hard part — deciding *which kind* of floating thing each candidate is. This is also why RFO/PLP recall measured through the full pipeline is lower than the model alone: a pixel has to clear the FDI gate before the model ever sees it.

Fail-loud design

If the ONNX file is missing or fails to load, the engine *can* fall back to a hand-written spectral rule set — but it **logs an error, banners the UI, and tags the result "heuristic"**, and with `REQUIRE_MODEL=1` it refuses to run at all. An un-benchmarked fallback must never be able to masquerade as a benchmarked model. Model loading is also behind a lock, so a second concurrent scan cannot catch a half-loaded state and silently degrade.

Reproduce any number in this document, live

```
# rebuild a dataset cache from raw GeoTIFFs
python -m ml.datasets.marida          python -m ml.datasets.mados
python -m ml.datasets.rfo             python -m ml.datasets.plp

# retrain on the GPU (writes candidates + a head-to-head verdict, promotes nothing)
conda run -n mpt-train python -m ml.train_perpixel_gpu \
    --epochs 80 --loss focal --weights cb --hidden 256,128,64,32 --calibrate

# independent exams
python -m ml.validate_plp             # deployed HDPE targets
python -m ml.validate_plp_clay        # + the Clay confirmation tier
```

The app itself carries an **AI Lab** page with an **Evaluate** button that re-scores the shipped ONNX model on a held-out split **live, in front of the panel** — macro-F1 first, accuracy captioned as water-inflated, confusion matrix drawn from the run.

Nothing on that screen is a hard-coded number.

SECTION 12

Panel crib sheet

Short, honest answers to the questions most likely to be asked.

QUESTION	ANSWER
Which model, in one sentence?	A 3-seed ensemble of 11→256→128→64→32→4 per-pixel MLPs, trained with focal loss, class-balanced weights and folded bias calibration, exported to ONNX with proven parity.
What is it trained on?	1,081,615 hand-labelled Sentinel-2 pixels from MARIDA (2022) and MADOS (2024) official splits, plus 3,789 points from four RFO scenes — 1,085,404 in total. Then 532,566 validation and 499,907 untouched test pixels.
How many datasets?	Five. MARIDA + MADOS + four RFO scenes train it; RFO's Durban and Marmara scenes and all of PLP are held-out exams; one synthetic baseline of our own that is never shipped.
How accurate?	Macro-F1 0.818. Plastic F1 0.799. In the HIGH-confidence tier that drives dispatch, plastic precision is 90 %. Accuracy is 0.99 but that is water-inflated and we always say so.
Why not a CNN?	We built one and it scored 0.555 versus 0.818 — it memorised scenes. The result is published in our model card, not hidden.
Isn't 0.99 accuracy suspicious?	Yes, and we flag it ourselves. 98 % of pixels are water. That is precisely why we lead with macro-F1 everywhere in the product.
You detect 0 of 33 real deployed targets.	Correct, and we publish it. They are 3.5–14 m targets in a 10 m pixel, so the pixel is ~90 % water. We tested the obvious fix and it made the product worse — plastic precision 0.776 → 0.660 for 2/33. On patch-scale benchmarks plastic recall is 90.6 % (MARIDA) and 89.9 % (MADOS).
How do we know you didn't tune on the test set?	Epoch selection and calibration both use the **validation** split. The test split is read once, at the end. And you can re-run the evaluation live from the AI Lab page.
What did you actually build?	Every line between the public datasets and the running product: the label mapping, four dataset adapters, the feature contract, the loss, the weighting, the calibration, the ensemble, the ONNX export and parity check, the two-tier confidence system, the Clay verifier, and the live inference path. Plus a from-scratch NumPy MLP with hand-written backpropagation and a hand-built ONNX graph, to demonstrate the fundamentals.
Biggest weakness?	Sub-pixel plastic. It is a sensor resolution limit at 10 m, and the fix is higher-resolution imagery or SAR fusion — a different project, not a better training run.

Three rules when presenting this

1. **Lead with macro-F1 (0.818)**, and caption accuracy as water-inflated every single time.
2. **Never round upward.** Say 0.82, not "about 0.9". Say 0/33, not "it struggles a bit".
3. **Volunteer the negative results** — the 0.555 CNN, the 0/33 PLP, the unvalidated drift ETAs. Every one of them has a measured explanation and a tested alternative attached. Hearing them from you reads as rigour; having them discovered reads as a gap.

Sources for every figure in this document: `python_engine/models/metrics.json` (published model card) · `perpixel_gpu_summary_ens.json` · `perpixel_gpu_summary_rfo.json` · `clay_head_summary.json` ·

`plp_validation.json` · `plp_clay_validation.json` · `fusion_thresholds.json`. Pixel counts read directly from the `.npz` caches; the headline accuracy, two-tier threshold table, RFO held-out figures and the PLP 0/33 result were re-measured against the shipped `marinenet.onnx` while this document was written.

One known staleness in the repo, noted for accuracy: the `trainSamples` field and `dataset` string inside `metrics.json` record only the MARIDA + MADOS count (1,081,615) and omit the 3,789 RFO training points that the shipped ensemble also learned from. The training-set composition quoted in this document is the one established by the RFO in/out ablation in `perpixel_gpu_summary_ens.json`, which is the reliable record. Held-out evaluation is unaffected either way — Durban and Marmara are withheld in both variants.

Marine Plastic Tracker · Model & Data Explainer · 30 July 2026